

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и
автоматизированных систем

Курсовой проект
по дисциплине: «Компьютерные сети»
тема: «Разработка приложения для доступа к облачным хранилищам данных с
поддержкой мульти аккаунта»

Автор работы _____ Дмитриев Андрей Александрович ПВ-223
(подпись)

Руководитель проекта _____ Федотов Евгений Александрович
(подпись)

Оценка _____

Белгород 2025

Содержание

Введение	3
1. Анализ предметной области и сравнение аналогов	4
2. Реализация серверной части	5
2.1 Выбор протокола передачи данных и языка программирования	5
2.2 Создание сервера.....	5
2.3 База-данных сервера	6
2.4 Обработка запросов клиента.....	7
2.5 Токены	8
3. Реализация клиентской части	9
3.1 Выбор инструментов и языка программирования.....	9
3.2 Конфигурация клиента	9
3.3 Процесс работы клиентской части	10
4. Примеры работы программы	13
Заключение	14
Список источников и литературы	15
Приложения	16

Введение

Данный курсовой проект посвящен разработке приложения для доступа к облачным хранилищам данных с поддержкой мульти аккаунта на Android.

Целью проекта является создание приложения, позволяющего добавлять, удалять и получать доступ к файлам с возможностью аутентифицироваться с различных аккаунтов, а также реализовать сервер-API – облачное хранилище данных.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Проектирование архитектуры программы:
 - a. Разработать схему взаимодействия между клиентом и сервером.
 - b. Определить протокол передачи данных.
 - c. Применить метод авторизации с помощью JWT.
2. Разработка серверной части на Spring Boot (Kotlin):
 - a. Создать сервер, принимающий команды от клиента.
 - b. Реализовать функции обработки команд.
 - c. Сконфигурировать фильтры безопасности и обработку JWT.
3. Разработка клиентского приложения на ОС Android (Compose, Kotlin):
 - a. Реализовать интерфейс со входом, регистрацией и списком файлов.
 - b. Используя средства Android, реализовать добавление и просмотр файлов.
 - c. Обработать все события, связанные с истечением срока жизни токенов и отключения сервера.
4. Проверка работоспособности приложения.

1. Анализ предметной области и сравнение аналогов

Приложение, использующее JWT (JSON Web Token) для авторизации и взаимодействия с облачным сервером, относится к клиент-серверным системам, где безопасность и управление доступом к данным играют ключевую роль. JWT представляет собой компактный токен, содержащий информацию о пользователе и его правах, подписанный секретным ключом. Приложение отправляет запросы к облачному серверу, который хранит файлы или данные, а сервер проверяет токен для предоставления доступа. Предметная область охватывает облачные хранилища, системы управления контентом (CMS), API-driven приложения и сервисы, требующие строгого контроля доступа, такие как Google Drive, Dropbox или корпоративные порталы.

OAuth 2.0. В отличие от JWT, который является самодостаточным токеном, OAuth 2.0 — это протокол авторизации, часто используемый для делегирования доступа. Например, Google Drive использует OAuth 2.0, где пользователь предоставляет доступ приложению через сторонний сервис (Google), получая токен доступа. OAuth сложнее в реализации, требует дополнительных запросов к серверу авторизации и refresh-токенов, но подходит для сценариев, где доступ делегируется между сервисами.

Session-based и API Key. Session-based авторизация, используемая в традиционных веб-приложениях, хранит состояние сессии на сервере, что требует значительных ресурсов при масштабировании, в отличие от stateless JWT, где вся информация содержится в токене. Например, системы вроде WordPress часто используют сессии, что усложняет горизонтальное масштабирование. API Key, применяемый в сервисах вроде AWS S3, прост в реализации, но менее безопасен, так как ключи статичны и не содержат данных о пользователе. JWT обеспечивает баланс между безопасностью и масштабируемостью, но требует осторожности при управлении сроком действия токенов и защитой от атак, таких как перехват.

2. Реализация серверной части

2.1 Выбор протокола передачи данных и языка программирования

Для взаимодействия между мобильным приложением и серверной частью был выбран протокол HTTP. Этот протокол является универсальным и ходовым, как и для организации API, так и для передачи файлом с помощью Multipart/form-data. Kotlin построен на языке Java, для которого есть библиотека Spring Boot, предоставляющая широкий функционал, как и при работе с HTTP, так и с безопасностью. Использована незащищённая версия протокола в целях упрощения реализации, в настоящем проекте следует не использовать данную версию протокола.

Для клиентской части также использован язык Котлин и Android SDK с библиотекой Retrofit для работы с API.

2.2 Создание сервера

Для сервера определены три группы эндпоинтов: «/auth» – для логина или регистрации, а также для обновления токенов; «/data», «/user» – для взаимодействия с данными и профилем пользователя (защищённые).

В Spring Boot защита достигается путём определения «фильтров безопасности». Для JWT необходимо проверять AccessToken при каждом обращении к защищённым эндпоинтам (Листинг 1).

```
val authHeader = request.getHeader("Authorization")
val tokenAccess =
    if (authHeader?.startsWith("Bearer ") == true)
        authHeader.substring("Bearer ".length)
    else
        throw NoTokenFoundException()

val login = try {
    jwtService.getLoginFromAccessToken(tokenAccess)
} catch (e: JwtException) {
    throw TokenAccessExpiredException()
} catch (e: SignatureException) {
    throw TokenBadSignatureException()
}

SecurityContextHolder.getContext().let {
    if (it.authentication == null) {
```

```

        it.authentication = UsernamePasswordAuthenticationToken(login, null,
emptyList())
    }
}

filterChain.doFilter(request, response)

```

Листинг 1. Фильтр для проверки JWT

Конфигурация представлена в листинге 2, в ней отключены некоторые проверки безопасности не относящиеся к API-серверу, добавлены кастомные фильтры и определены группы эндпоинтов с разграничением доступа.

```

@Bean
fun filterChain(http: HttpSecurity): SecurityFilterChain = http
    .csrf { it.disable() }
    .cors { it.disable() }
    .sessionManagement {
it.sessionCreationPolicy(SessionCreationPolicy.STATELESS) }
    .authenticationProvider(authenticationProvider())
    .authorizeHttpRequests {
        it.requestMatchers("/auth/**").permitAll()
        .requestMatchers("/user/**").authenticated()
        .requestMatchers("/data/**").authenticated()
        .anyRequest().permitAll()
    }
    .httpBasic(withDefaults())
    .addFilterBefore(exceptionHandlingFilter, LogoutFilter::class.java)
    .addFilterBefore(jwtRequestFilter,
UsernamePasswordAuthenticationFilter::class.java)
    .build()

```

Листинг 2. Конфигурация сервера

2.3 База-данных сервера

Используется база данных с двумя таблицами: для пользователей и связанная с ней таблица файлов (Рисунок 1). В виде СУБД используется PostgreSQL, так как построена на общепринятой реляционной модели и подходит для созранения файлов. Для наглядности данные не шифруются, хотя это можно достаточно просто сделать, дополнив код реализованных сервисов.

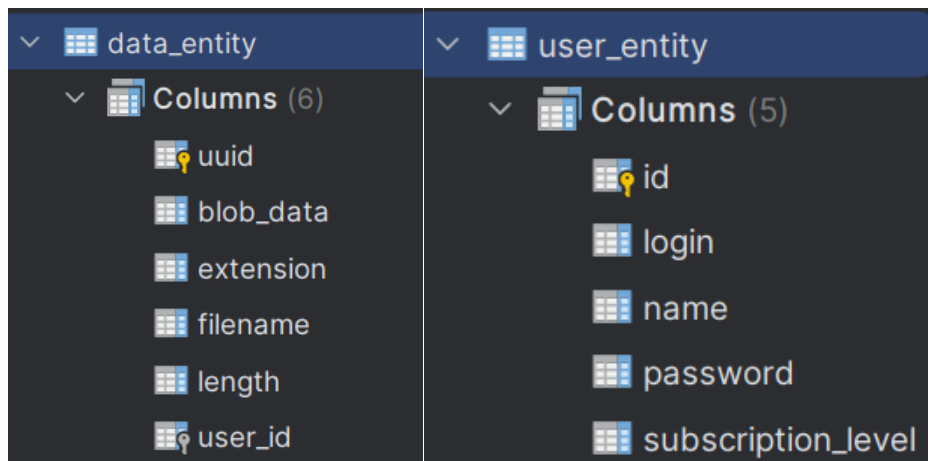


Рисунок 1. Структура таблиц

2.4 Обработка запросов клиента

Эндпоинты группы «/auth» направляют post-запросы с json в теле, это предназначено для повышения безопасности за счёт особенности работы запроса.

Остальные запросы требуют указания AccessToken в заголовке: «Authorization Bearer <token>». Если токен неактуальный, то запрос может отклоняться с кодами 401, 403 и 404.

Передача файла (Листинг 3). Для получения данных используется стандартный для таких задач MIME-тип файла, как application/octet-stream. Для отправки используется multipart/form-data, чтобы не только получить поток байтов, но и передать метаданные.

```
@GetMapping("/{uuid}")
fun getFileByUuid(@PathVariable uuid: String): ResponseEntity<Resource> {
    val data = dataService.getData(login, uuid) ?: return
    ResponseEntity(HttpStatus.NOT_FOUND)

    val resource: Resource = ByteArrayResource(data.blobData)
    val headers = HttpHeaders().apply {
        contentType = MediaType.APPLICATION_OCTET_STREAM
        contentDisposition = ContentDisposition.attachment()
            .filename(data.filename, StandardCharsets.UTF_8)
            .build()
    }

    logger.info("Getting data by uuid: $uuid")
    return ResponseEntity
```

```

        .ok()
        .headers(headers)
        .body(resource)
    }

@PostMapping(consumes = [MediaType.MULTIPART_FORM_DATA_VALUE])
fun save(
    @RequestParam file: MultipartFile,
    @RequestParam filename: String,
): ResponseEntity<DataResponseDto> {
    val request = DataRequestDto(
        filename = filename,
        blob = file.bytes
    )
    val resultOfSave = dataService.saveData(request, login)
    logger.info("Saving data for user: $login")
    return ResponseEntity(resultOfSave, HttpStatus.OK)
} // blob = null

```

Листинг 3. Эндпоинты для сохранения и получения файлов

2.5 Токены

Как было сказано выше, токены в проекте играют основную роль для поддержания актуальности прав пользователя и лучшего пользовательского опыта.

После регистрации пользователь получает refresh и access token. Первый «живёт» долго и нужен для обновления access токена, пока время жизни не истекло пользователю не нужно выполнять вход со своим логином и паролем. Второй token «живёт» меньше и передаётся в заголовке, где валидируется фильтром безопасности (Рисунок 2).

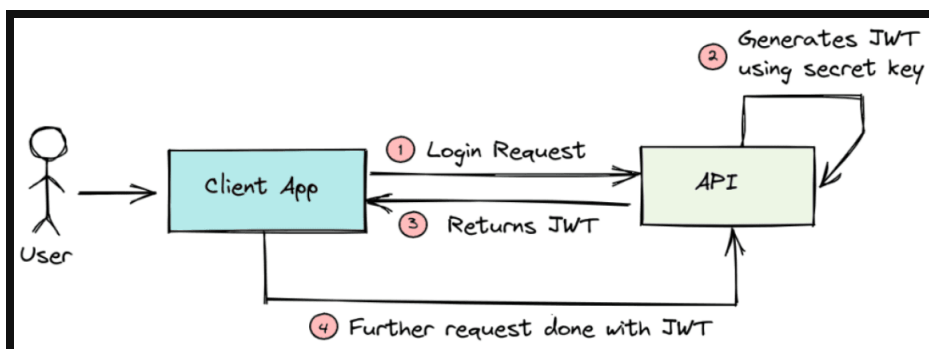


Рисунок 2. Схема работы JWT.

JWT имеет некоторые особенности в своей работе. Он может содержать беспрепятственно декодируемую информацию с подписью. В текущей реализации в «полезной нагрузке» передаётся логин и устанавливается время жизни, которое автоматически проверяется при парсинге библиотекой jjwt.

3. Реализация клиентской части

3.1 Выбор инструментов и языка программирования

В качестве языка для клиентской части был выбран язык Kotlin. Это обуславливается целью проекта – создание приложения на ОС Android.

В качестве инструмента для реализации было выбрано IDE Android Studio – основное IDE для работы с Android. Оно также включает в себя эмулятор различных Android устройств. В данном проекте используется эмулятор Google Pixel 9, 34 API (Android 14).

Для работы с API используется Retrofit, так как он предоставляет удобную настройку и описание эндпоинтов.

3.2 Конфигурация клиента

Конфигурация представляет собой клиента, с прописанными таймаутами, и настройку объекта библиотеки, а именно указание заголовка «Authorization» и конверторы для запросов и ответов (Листинг 4).

Эндпоинты повторяют эндпоинты с сервера только в виде интерфейса.

```

val okHttpClient = OkHttpClient.Builder()
    .addInterceptor(AuthInterceptor(tokenDao))
    .addInterceptor(LoggingInterceptor())
    .connectTimeout(30, TimeUnit.SECONDS)
    .readTimeout(60, TimeUnit.SECONDS)
    .writeTimeout(60, TimeUnit.SECONDS)
    .build()

return Retrofit.Builder()
    .baseUrl(BASE_URL)
    .client(okHttpClient)
    .addConverterFactory(ScalarsConverterFactory.create())
    .addConverterFactory(GsonConverterFactory.create())
    .build()

```

Листинг 4. Конфигурация Retrofit

3.3 Процесс работы клиентской части

В начале работы приложения открывается экран с файлами, если с запросов приходят 401, 403 или 404 ошибки, то происходит переход на страницу логина. Происходит это с помощью внешнего обработчика токенов (Листинг 5).

```

suspend operator fun invoke(
    exception: Exception,
    tokenHandlerHolder: TokenHandlerHolder
) = withContext(Dispatchers.IO) {
    try {
        if (exception is HttpException && exception.code().let { it == 401 || it == 403 }) {
            tokenDao.tokenAccess =
            authEndpoints.refreshAccessToken(tokenDao.tokenRefresh)
            return@withContext
        }
    } catch (e: HttpException) {
        if (e.code().let { it == 401 || it == 403 || it == 404 }) {
            withContext(Dispatchers.Main) {
                tokenHandlerHolder.onRefreshTokenExpired()
            }
            return@withContext
        }
    }

    withContext(Dispatchers.Main) {
        tokenHandlerHolder.onAnotherException()
    }
}

```

Листинг 5. Обработчик токенов

Со страницы логина можно переходить на страницу регистрации с переносом введенных логина и пароля (Рисунок 3).

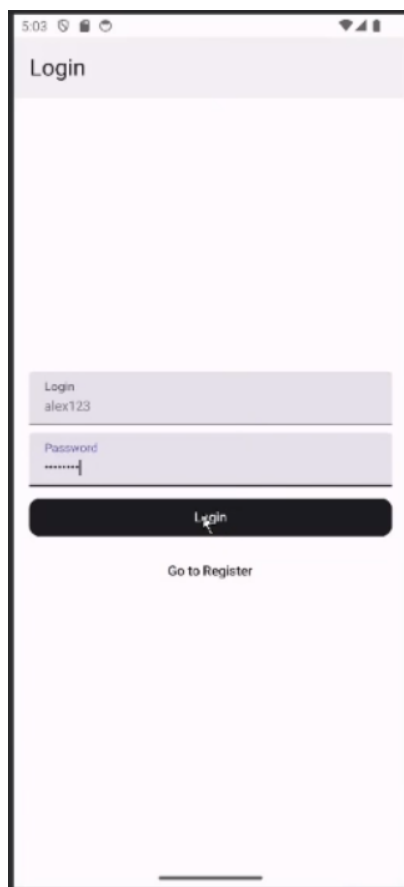


Рисунок 3. Форма логина.

После аутентификации происходит передача токенов для авторизации, которые сохраняются на клиенте и используются в функции обработки токенов.

На экране с файлами отображаются карточки с названием, размером и иконкой, которая зависит от расширения файла. При одиночном нажатии на карточку открывается файл стандартным средством, определённым в системе. При удержании файл удаляется с сервера. При нажатии на «плавающую» клавишу с иконкой «+» можно добавить файл средствами Android. В верхнем углу кнопка стирает токены из памяти и возвращает на форму логина.



Рисунок 4. Форма файлов.

4. Примеры работы программы

Далее, приведен пример работы с несколькими аккаунтами.

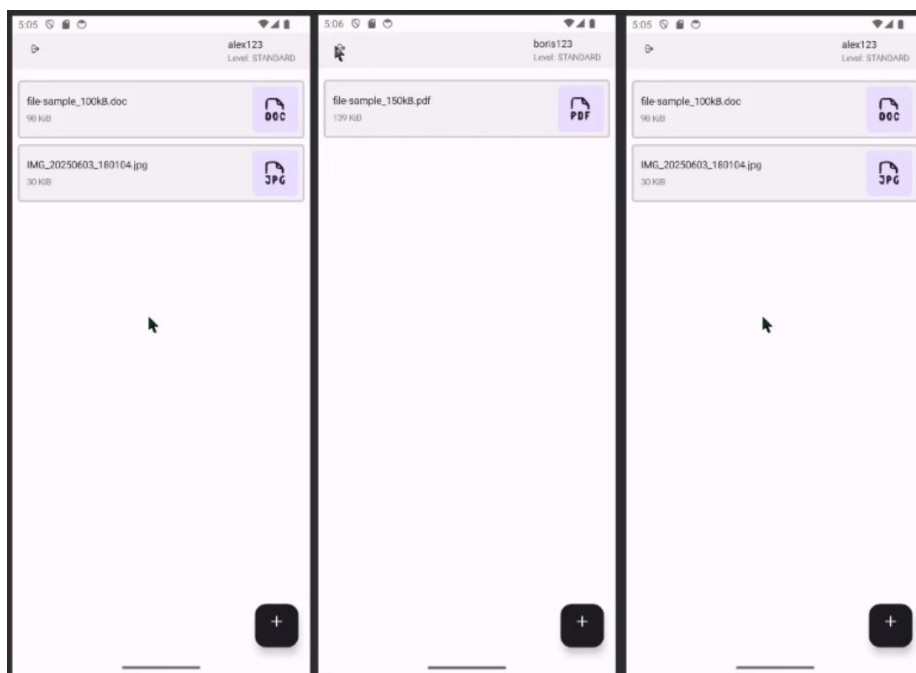


Рисунок 5. Пример работы с несколькими аккаунтами.

Заключение

В процессе выполнения курсовой работы было созданы клиент и сервер для взаимодействия с удалённым облаком файлов, поддерживающим несколько аккаунтов. Для реализации серверной части использовалась библиотека Spring Boot. Для реализации клиентской части использовался Android SDK с библиотекой Retrofit. В ходе работы были получены навыки работы с языками Kotlin, JWT авторизацией и перечисленными библиотеками. Приобретён опыт разработки клиент-серверного приложения для ОС Android, а также получены знания о работе HTTP протокола.

Список источников и литературы

1. Документация Spring Boot [Электронный ресурс] Режим доступа: <https://docs.spring.io/spring-boot/index.html>
2. Документация Kotlin [Электронный ресурс] Режим доступа: <https://kotlinlang.org/docs/home.html>
3. Статья Habr «Изучаем Retrofit 2» [Электронный ресурс] Режим доступа: <https://habr.com/ru/articles/314028/>
4. Документация Android Compose [Электронный ресурс] Режим доступа: <https://developer.android.com/develop/ui/compose/documentation>

Приложения

Исходный код проекта расположен на GitHub.

Ссылка на GitHub, сервер: <https://github.com/AnDreV133/FileCloudApi>

Ссылка на GitHub, клиент: <https://github.com/AnDreV133/FileCloudApp>